Meta-Learning for Automated Anomaly Detection Model Selection

1st Lucien McNulty
Computer Science Department
Loyola Marymount University
Los Angeles, California

***Abstract*—Anomaly detection is crucial across domains like finance (fraud) and cybersecurity (intrusion detection). However, diverse datasets (size, noise, skewness, etc.) require repeated testing of multiple algorithms, which is time-consuming and inefficient. This project aims to develop a meta-learning framework that automates the process of selecting and configuring anomaly detection models for diverse datasets. 58 datasets from the Numenta Anomaly Benchmark (NAB) were used as individual tasks (traditional machine learning involves training a model on a single specific task: meta-learning involves learning across a collection tasks). Five unsupervised anomaly detection models were evaluated on each dataset: Isolation Forest, PCA reconstruction error, One-Class SVM, an autoencoder, and K-means clustering. Then, for every dataset, a set of statistical/temporal meta-features were extracted and a performance matrix for each model's F1 scores was constructed. The meta-learner based on Random Forest is trained to map dataset meta-features to the index of the best-performing base model. Using leave-one-dataset-out cross-validation, the meta-learner achieved an accuracy of 44.8% in predicting the best model among the five candidates, which substantially outperforms a 20% random baseline. These results demonstrate the feasibility of meta-learning for automated anomaly detector selection, while also highlighting limitations of simple meta-features and unsupervised evaluation in highly imbalanced settings.**

I. Introduction

Anomaly detection underpins many real-world monitoring systems: fraud detection in credit card transactions, intrusion detection in cybersecurity, performance monitoring for cloud services, traffic flow monitoring, and more. All these domains deal with time-series datasets of different sampling rates, noise characteristics, and anomaly patterns. As a result, no single anomaly detection algorithm works best across all datasets. In practice, data scientists often cycle through models and tune hyperparameters manually, tediously comparing metrics: this is an expensive process when new datasets arrive frequently.

This project explores whether meta-learning can reduce that overhead by learning how to choose an anomaly detection model based on properties of the dataset. I focus exclusively on time-series data and treat each time-series dataset as a separate task for the meta-learner. For each dataset, I extract meta-features that summarize its distribution and temporal structure, evaluate several unsupervised anomaly detection models, and then train a meta-learner that predicts which model will perform best.

Initially, I planned to use supervised models for anomaly detection, but the NAB datasets have very few points labeled as anomalous relative to normal points. This extreme class imbalance makes supervised learning fragile, motivating the use of unsupervised anomaly detectors that model "normal" behavior and flag deviations as anomalies instead. The five base models ultimately used were: Isolation Forest, PCA reconstruction error, One-Class SVM, an autoencoder, and K-means clustering.

Contributions/Key Findings

- Constructed a full meta-learning pipeline for time-series anomaly detector selection
- Benchmarked 5 unsupervised anomaly detectors across 58 NAB datasets
- Demonstrated that simple statistical and temporal meta-features can predict the best model with 44.8% LOOCV accuracy
- Evidenced the limitations imposed by class imbalance and fixed score thresholding.

II. Data Description

A. Numenta Anomaly Benchmark (NAB)

The experiment was conducted on 58 time-series datasets from the Numenta Anomaly Benchmark (NAB), a widely used benchmark for streaming anomaly detection. Initially, datasets from more sources outside NAB were considered, but only NAB was used to keep the scope focused as the characteristics of datasets from different disciplines may create too much variance for the metalearner to learn. NAB provides labeled time-series for several domains, including:

- `artificialNoAnomaly`: Synthetic data with no true anomalies
- `artificialWithAnomaly`: Synthetic data with injected anomalies
- `realAdExchange`: Online advertising metrics
- `realAWSCloudwatch`: AWS system metrics
- `realKnownCause`: Real datasets with documented anomaly causes
- `realTraffic`: Highway sensor data
- `realTweets`: Twitter volume for major companies

Each dataset consists of a timestamp column (interval where anomalies occur) and a single value column. The timestamp intervals were converted into binary labels for evaluation (0 or 1 depending on whether the observation fell within an anomaly window).

B. Basic Statistics

After loading and cleaning all CSV files from the NAB directory, I obtain:

- Number of datasets: 58
- Typical series length: approximately 4,000–20,000 time points for many datasets
- Anomaly ratios: for many real datasets, the fraction of points labeled anomalous is on the order of 5–10%. For synthetic "no anomaly" series, the anomaly ratio is exactly 0. Both the count of anomaly points (label = 1) and normal points (label = 0) are tracked.

These statistics highlight preliminary key challenges: anomalies are rare and some datasets have no labeled anomalies at all.

C. Preprocessing Pipeline

Each time-series dataset undergoes a standardized preprocessing pipeline prior to model evaluation. First, all raw NAB CSV files are cleaned to ensure consistent formatting, valid timestamps, and the removal of duplicate or missing records. For each dataset, timestamps are explicitly converted to datetime format to ensure proper temporal alignment.
NAB's anomaly windows are converted into binary point-level labels such that all points are first assigned label 0 (normal), then points whose timestamp is within the anomaly window are reassigned label 1 (anomaly).
After labeling, the final processed dataset contains three aligned components for every timestamp: the cleaned value signal, a timestamp index, and a binary anomaly label. These labeled time series are then passed to each unsupervised base anomaly detection model for scoring and evaluation.

III. Methodology

The overall pipeline follows a meta-learning structure as follows:

1) **Base model evaluation:** Each dataset is processed by every base model to collect per-dataset F1 scores and construct a performance matrix.
2) **Meta-feature extraction:** Descriptive and temporal features are computed to characterize each dataset.
3) **Meta-learner training:** Meta-features are used as inputs and the index of the best-performing base model is used as the target variable.
4) **Meta-evaluation:** Leave-one-dataset-out cross-validation is used to estimate how well the meta-learner predicts the best model for unseen datasets.

A. Base Models:

Isolation Forest: Isolation forest: very similar to random forest. Random forests optimize splits to predict labels, while isolation forests use random splits to isolate anomalies quickly: isolation forest is to anomaly detection as random forest is to prediction. anomalies are easier to separate from the rest of data, so they require fewer splits in a random tree. I used a standard scikit-learn implementation to compute an anomaly score for each time point.

PCA Reconstruction Error: For PCA reconstruction, I standardized the value series, fit PCA on the entire time series, projected the data to a lower dimensional subspace using only the top principal components, then reconstructed the data back to the original space: the reconstruction error was computed for each point.

One-Class SVM: The One-Class SVM draws a smooth boundary enclosing normal data and flagging outside points as abnormal. Its hyperparameters are as follows: kernel: shape of boundary, Gamma: flexibility of boundary, Nu: % of anomalies allowed (tightness of boundary). I used a scikit-learn implementation of One-Class SVM on standardized values and obtained a continuous anomaly score via the model's decision function; Points with the most extreme 1% of scores were classified as anomalies.

Autoencoder: The autoencoder is a shallow neural network trained to reconstruct sliding windows of the normalized time-series. It has the following specifications: window size = 20, hidden dimension size = 32, bottleneck dimension size = 8, 40 epochs, batch size =128, and learning rate = e^{-3} that halves at select epochs simple schedule halves the learning rate at select epochs.
After training, each sliding window is reconstructed, from which their reconstruction error becomes the anomaly score, assigning each window's error to its corresponding time point.

K-Means Clustering: K-means clusters the standardized time series into three groups (k = 3). K was fixed at 3 for consistency, although dataset-adaptive k-selection could improve performance. Each point's distance to its assigned cluster center is used as its anomaly score (larger distances indicate anomalous behavior).

B. Scores to Binary Predictions

Binary anomaly labels are obtained by thresholding each model's scores at the 99th percentile.

C. Evaluation Metric

I chose to evaluate each model with an F1 score instead of AUC since anomalies were so rare. Precision, recall, and F1 for every dataset-model pair were stored in a performance matrix where each row represents a dataset and each column represents a base model (58 X 5).

D. Random Forest Meta-Learner Training

For each dataset, the index of the base model with the highest F1 score is used as the meta-label. The meta-learner is trained to map dataset meta-features to this best-model index. The meta-learner was trained on a Random Forest classifier as it performs well on a low number of samples, is resistant to overfitting (can model nonlinear meta-feature interactions while remaining resistant to overfitting via ensembling), and is robust to noisy targets (F1 scores). Performance is evaluated using leave-one-dataset-out cross-validation, where the model is trained on 57 datasets and tested on the leftout dataset. The LOOCV accuracy is 44.8%, exceeding the 20% random baseline.

IV. Experiment and Results

Mean F1 score for base models: The F1 scores across all models is relatively low: this reflects the difficulty of point-level anomaly detection with a fixed top-1% threshold, not the failure of the models themselves. PCA reconstruction performs best on average with the autoencoder only slightly behind. Datasets with no anomalies yielded an F1 of 0 for all models since any predicted anomalies became false positives.

TABLE I
MEAN F1 SCORES OF BASE ANOMALY DETECTION MODELS ACROSS 58 NAB DATASETS.

Model	Mean F1
PCA reconstruction	0.085
One-Class SVM	0.068
Isolation Forest	0.060
K-means clustering	0.052
Autoencoder	0.084

TABLE II
CONFUSION MATRIX OF THE META-LEARNER (ROWS = TRUE BEST MODEL, COLUMNS = PREDICTED BEST MODEL).

True \ Pred	iso_f1	pca_f1	ocsvm_f1	kmeans_f1	ae_f1
iso_f1	10	5	1	1	0
pca_f1	3	12	0	0	4
ocsvm_f1	1	5	0	0	2
kmeans_f1	1	1	0	0	0
ae_f1	3	4	1	0	4

The confusion matrix shows that the meta-learner predicts Isolation Forest and PCA reconstruction the best, which aligns with the strongest average base-model F1 scores. However, the model fails to correctly identify OCSVM and K-means as best-performing models in any dataset, suggesting limited discriminative power of the meta-features for these classes.

TABLE III
CLASSIFICATION REPORT OF THE META-LEARNER ACROSS BASE MODELS.

Model	Precision	Recall	F1-score	Support
iso_f1	0.56	0.59	0.57	17
pca_f1	0.44	0.63	0.52	19
ocsvm_f1	0.00	0.00	0.00	8
kmeans_f1	0.00	0.00	0.00	2
ae_f1	0.40	0.33	0.36	12
Accuracy	–	–	0.45	58
Macro Avg	0.28	0.31	0.29	58
Weighted Avg	0.39	0.45	0.41	58

While the meta-learner achieves its highest performance on Isolation Forest and PCA, it fails to correctly predict OCSVM and K-means as optimal models. This reflects their weak average F1 performance and class-imbalance/weak separability in the meta-feature space for these classes.

A. Meta-Learner Accuracy

Using leave-one-dataset-out cross-validation, the metalearner achieved a baseline accuracy of 44.8% (LOOCV accuracy), which is over double the random baseline of 20%. Given the noisy F1 estimates and small sample size (of 58 tasks), this level of performance is unsurprising.

V. Conclusion and Future Work

While the accuracy scores weren't ideal, this project acts as a great proof of concept for cross-domain meta-learning in anomaly detection. The final meta-learner achieved a 44.8% leave-one-dataset-out accuracy in predicting the best anomaly detector among the 5 candidate models, substantially better than chance. This suggests that even small sets of generic meta-features encode predictive information to use on new time-series datasets. However, there are several limitations: limited meta-features, lack of hyperparameter meta-learning, label noise/imbalance, and a small number of tasks. Future work could address these limitations, through richer meta-features, multi-metric meta-learning (use more criteria than just F1 to pick the best model), hyperparameter meta-learning, more complex meta-learner architecture, and additional base models. If I were to focus on a single new addition, I would want to include datasets from different disciplines, testing to see how well the metalearner performs (examine how the importance of dataset characteristics are overshadowed by the type of discipline).